Тема 11. Работа с данными и API

хекслет колледж

Цель занятия:

Научить сервер принимать информацию от клиента, обрабатывать её и отдавать структурированный ответ (JSON), понятный веб - приложению.

хекслет колледж

Учебные вопросы:

1. Роутинг (Маршрутизация) на нативном Node.js
2. GET-запросы и параметры (Query Strings)
3. POST-запросы: Прием больших данных
4. Работа с JSON (Стандарт обмена данными)
5. Создание простейшего API (Практика)

хекслет колледж

1. Роутинг (Маршрутизация) на нативном Node.js

Роутинг - это механизм определения того, какой код должен выполниться в зависимости от URL-адреса (req.url) и метода запроса (req.method).

хекслет колледж

Как сервер понимает, куда пришел пользователь?

Когда мы вводим в браузере localhost:3000/contacts, Node.js получает объект req, где свойство req.url будет содержать строку "/contacts".
Наша задача - проанализировать эту строку.

хекслет колледж

Простейший роутер (через if или switch)

В нативном Node.js (без Express) мы создаем маршруты вручную.
Самый понятный способ - конструкция switch.

хекслет колледж

```
const http = require('http');

const server = http.createServer((req, res) => {
    // 1. Извлекаем адрес из запроса
    const path = req.url;

    // 2. Устанавливаем общий заголовок для всех ответов
    res.setHeader('Content-Type', 'text/html; charset=utf-8');
```

хекслет колледж

```
    // 3. Распределяем логику (Роутинг)
    switch (path) {
        case '/':
            res.statusCode = 200;
            res.end('<h1>Главная страница</h1>');
            break;

        case '/users':
            res.statusCode = 200;
            res.end('<h1>Список студентов</h1><ul><li>Иван</li><li>Мария</li></ul>');
            break;

        case '/api/status':
            // Пример ответа для мобильного приложения
            res.setHeader('Content-Type', 'application/json');
            res.end(JSON.stringify({ status: 'OK', uptime: '100%' }));
            break;

        default:
            // Если адрес не совпал ни с одним из вышеуказанных
            res.statusCode = 404;
            res.end('<h1>404: Страница не найдена</h1>');
            break;
    }
});
```

екслет колледж

```
// 3. Заставляем сервер "слушать" определенный порт
const PORT = 3000;
server.listen(PORT, () => {
    console.log(`Сервер запущен! Адрес: http://localhost:${PORT}`);
    console.log('Нажмите Ctrl+C, чтобы остановить его.');
});
```

хекслет колледж

Почему это важно для API?

Для приложения роутинг - это способ обращаться к разным функциям вашего бэкенда.

- GET /tasks - получить список дел.
- GET /profile - получить данные пользователя.

Для приложения это выглядит как разные «ручки» (endpoints) одного и того же сервера.

хекслет колледж

Проблема «грязного» URL

Важный момент: req.url содержит всё, что идет после порта.

Если пользователь введет /users?id=5, наш switch не сработает, потому что он ищет строгое совпадение с "/users".

В нативном Node.js адрес и его параметры перемешаны в одной строке. Чтобы роутинг работал стабильно, нам нужно научиться отделять «путь» (куда идем) от «параметров» (что несем).

хекслет колледж

Итог:

- **Роутинг** - это «разводка» запросов по разным функциям.
- Основа роутинга в Node.js - объект req.url.
- Блок default - всегда должен возвращать 404 ошибку, чтобы клиент не висел в ожидании.
- В реальных проектах роутинг выносят в отдельные файлы, чтобы код сервера не превращался в бесконечный список if/else.

хекслет колледж

2. GET-запросы и параметры (Query Strings)

Когда вы ищете что-то в Google, ваш запрос отображается прямо в адресной строке: google.com/search?q=nodeJS. Всё, что идет после знака ?, называется Query String (строка запроса).

хекслет колледж

Анатомия URL с параметрами

http://localhost:3000/products?category=phones&limit=10

/products - Путь (Path). Куда мы идем.

? - Разделитель. Говорит: «Дальше пошли данные».

category=phones - Первый параметр (ключ=значение).

& - Соединитель. Позволяет передать несколько параметров.

limit=10 - Второй параметр.

хекслет колледж

Проблема в Node.js

Как мы уже выяснили, req.url выдает всё одной кучей: "/products?category=phones&limit=10". Делить эту строку вручную через .split('?') - плохая практика, потому что параметры могут быть закодированы или идти в разном порядке.

Современный способ: Объект URL

Для удобного разбора параметров в Node.js используется встроенный класс URL.

хекслет колледж

```
const server = http.createServer((req, res) => {
    // 1. Создаем объект URL (вторым аргументом передаем хост для корректной сборки)
    const fullUrl = new URL(req.url, `http://${req.headers.host}`);

    // 2. Получаем чистый путь без параметров (например, "/products")
    const pathname = fullUrl.pathname;

    // 3. Получаем доступ к параметрам через searchParams
    const category = fullUrl.searchParams.get('category'); // "phones"
    const limit = fullUrl.searchParams.get('limit');       // "10"

    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });

    if (pathname === '/search') {
        res.end(`<h1>Результаты поиска</h1>
                 <p>Категория: ${category}</p>
                 <p>Показать товаров: ${limit}</p>`);
    } else {
        res.end('Используйте /search?category=...&limit=...');
    }
});
```

екслет колледж

Зачем это нужно в приложении?

Представьте, что ваше приложение отображает список товаров.

- Пользователь выбирает фильтр «Электроника».
- Приложение отправляет запрос: GET /products?type=electronics.
- Сервер вытаскивает слово electronics, идет в базу данных и отдает только нужные товары.

хекслет колледж

Важные нюансы:

- Типы данных: Все параметры из searchParams приходят как строки. Если вам нужно число (например, для limit), не забудьте сделать Number(limit).

- Безопасность: Никогда не передавайте через GET-параметры пароли или секретные данные. Они сохраняются в истории браузера и в логах сервера. Для этого есть POST-запросы.

хекслет колледж

Итог:

- Query String - это способ передать серверу мелкие настройки запроса через URL.
- Параметры начинаются с ? и разделяются &.
- В Node.js для разбора параметров используем new URL(req.url, ...).
- Свойство searchParams.get('ключ') - самый быстрый способ достать значение.

кекслет колледж

3. POST-запросы: Прием больших данных

Если GET-запрос - это «открытка», где всё написано на виду, то POST-запрос - это «посылка».

Данных в ней может быть очень много (например, фотография или длинный текст), поэтому Node.js не ждет, пока посылка дойдет целиком, а начинает обрабатывать её частями.

хекслет колледж

Почему данные приходят «кусками»?

Представьте, что вам присылают файл размером 100 МБ.
Если Node.js будет ждать полной загрузки в оперативную память перед обработкой, сервер «умрет», когда придет 10 таких пользователей.
Поэтому Node.js использует событийную модель: он говорит: «О, прилетела порция байтов! Держи её. О, еще одна!».

хекслет колледж

Как собрать данные (События data и end)

Для работы с телом POST-запроса нам нужно прослушать два события у объекта req:

- req.on('data', chunk => { ... }) - срабатывает, когда пришла очередная порция (чанк) данных.
- req.on('end', () => { ... }) - срабатывает, когда передача данных полностью завершена.

кекслет колледж

Пример кода: Прием данных от клиента

```
const http = require('http');

const server = http.createServer((req, res) => {
    if (req.method === 'POST') {
        let body = ''; // Сюда будем складывать кусочки данных

        // 1. Слушаем порции данных
        req.on('data', chunk => {
            // chunk — это Buffer (байты), превращаем его в строку и добавляем в body
            body += chunk.toString();
        });
```

кекслет колледж

```
        // 2. Когда данные закончились
        req.on('end', () => {
            console.log('Данные получены:', body);

            res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
            res.end(`Сервер получил ваши данные: ${body}`);
        });

    } else {
        res.writeHead(405);
        res.end('Используйте POST запрос');
    }
});

server.listen(3000);
```

екслет колледж

Проверяем:

```
C:\Users\user>curl -X POST -d "Hello, Server!" http://localhost:3000
Сервер получил ваши данные: Hello, Server!
C:\Users\user>
```

хекслет колледж

Или в Postman:

http://localhost:3000/

Save | Share

POST http://localhost:3000/ Send

Params | Authorization | Headers (8) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

```
1  Hello!
```

Body | Cookies | Headers (5) | Test Results | 200 OK | 46 ms | 229 B

Raw | Preview | Visualize

```
1  Сервер получил ваши данные: Hello!
```

екслет колледж

Важно:

- Буферы (Buffers): chunk - это не строка, а массив байтов. Мы делаем .toString(), потому что ожидаем текст (JSON или форму).
- Безопасность (Anti-Flood): В реальных серверах стоит ограничитель. Если body.length превышает, допустим, 1 МБ, соединение нужно обрывать, чтобы серверу не переполнили память вредоносным огромным запросом.
- Синхронность: Важно подчеркнуть, что res.end() должен находиться внутри обработчика end. Если написать его снаружи, сервер ответит клиенту еще до того, как данные дойдут.

хекслет колледж

Итог:

- POST используется для передачи чувствительных или больших данных.
- Данные в Node.js передаются потоком (кусками).
- Используем событие data для накопления данных и end для финальной обработки.
- Объект req сам по себе является «читаемым потоком» (Readable Stream).

хекслет колледж

4. Работа с JSON (Стандарт обмена данными)

JSON (JavaScript Object Notation) - это текстовый формат обмена данными. Он выглядит как объект JavaScript, но технически это просто строка.

Чтобы сервер мог работать с этими данными, нам нужно выполнить два действия:

- Парсинг (Разбор): Превратить входящую JSON-строку от клиента в JS-объект.
- Сериализация: Превратить наш JS-объект в JSON-строку для отправки отв

кекслет колледж

А. Прием JSON (Парсинг)

Когда данные приходят через POST, они прилетают в виде сырого текста. Чтобы достать из них значения, мы используем JSON.parse().

хекслет колледж

```
req.on('end', () => {
    try {
        // Превращаем строку в объект
        const data = JSON.parse(body);
        console.log('Имя пользователя:', data.username);
    } catch (err) {
        // Если клиент прислал кривой JSON, сервер не должен упасть
        res.statusCode = 400;
        res.end('Ошибка: Некорректный JSON');
    }
});
```

кекслет колледж

Б. Отправка JSON (Ответ)

Чтобы клиент (например, ваш смартфон) понял, что вы прислали ему не просто текст, а структурированные данные, нужно:

- Указать заголовок Content-Type: application/json.
- Превратить объект в строку через JSON.stringify().

хекслет колледж

Полный пример API-обработчика

```
const http = require('http');

const server = http.createServer((req, res) => {
    if (req.method === 'POST' && req.url === '/api/user') {
        let body = '';

        req.on('data', chunk => { body += chunk.toString(); });

        req.on('end', () => {
            // 1. Парсим входящие данные
            const userData = JSON.parse(body);

            // 2. Логика (например, "создаем" ID для пользователя)
            const responseData = {
                message: 'Пользователь успешно создан',
                userId: Math.floor(Math.random() * 1000),
                receivedData: userData
            };
```

екслет колледж

```
            // 3. Отправляем JSON-ответ
            res.writeHead(201, { 'Content-Type': 'application/json; charset=utf-8' });
            res.end(JSON.stringify(responseData));
        });
    }
});

server.listen(3000);
```

хекслет колледж

Проверка:

POST http://localhost:3000/api/user Send

Params Authorization Headers (8) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "username": "Alex"
3 }
```

Body Cookies Headers (5) Test Results 201 Created 10 ms 297 B

{} JSON Preview Visualize

```
1 {
2     "message": "Пользователь успешно создан",
3     "userId": 707,
4     "receivedData": {
5         "username": "Alex"
6     }
7 }
```

кекслет колледж

Схема взаимодействия (Frontend + Backend):

- Браузер (Frontend): Скрипт на странице (например, при нажатии кнопки «Добавить») отправляет запрос fetch методом POST с телом {"title": "Купить хлеб"}.
- Node.js (Backend): Принимает данные как поток байтов, собирает их в строку, превращает в объект и сохраняет (в массив или базу данных).
- Node.js (Backend): Формирует ответный объект {"status": "success", "id": 1}, превращает его в JSON-строку и отправляет обратно с кодом 201 Created.
- Браузер (Frontend): Получает ответ, парсит JSON и с помощью JavaScript динамически обновляет страницу (добавляет новый элемент в список задач без перезагрузки).

кекслет колледж

Итог:

- JSON.parse(string) - превращает текст в объект (используем при приеме).
- JSON.stringify(object) - превращает объект в текст (используем при ответе).
- Try-Catch - обязателен при парсинге, так как любая лишняя запятая в JSON вызовет фатальную ошибку сервера.
- Заголовок application/json - обязательный маркер для современных API.

кекслет колледж

5. Создание простейшего API (Практика)

На практике API (Application Programming Interface) для веб-приложения - это набор «ручек» (endpoints), к которым фронтенд обращается за данными.

Мы создадим простейший менеджер задач (To-Do List).

хекслет колледж

Архитектура нашего мини-проекта:

- Хранилище: Массив в оперативной памяти (временно, до изучения БД).
- Метод GET /tasks: Отдает весь список задач.
- Метод POST /tasks: Добавляет новую задачу в список.

хекслет колледж

Полный код сервера API

```
const http = require('http');

// Наша имитация базы данных
let tasks = [
    { id: 1, title: 'Выучить основы Node.js', completed: false }
];

const server = http.createServer((req, res) => {
    // Устанавливаем заголовок JSON для всех ответов
    res.setHeader('Content-Type', 'application/json; charset=utf-8');

    // --- МАРШРУТИЗАЦИЯ ---

    // 1. Получение всех задач
    if (req.url === '/tasks' && req.method === 'GET') {
        res.statusCode = 200;
        res.end(JSON.stringify(tasks));
    }
```

екслет колледж

```
// 2. Добавление новой задачи
else if (req.url === '/tasks' && req.method === 'POST') {
    let body = '';

    req.on('data', chunk => {
        body += chunk.toString();
    });

    req.on('end', () => {
        try {
            const data = JSON.parse(body);

            // Валидация: проверим, прислал ли пользователь текст задачи
            if (!data.title) {
                res.statusCode = 400;
                return res.end(JSON.stringify({ error: 'Поле title обязательно' }));
            }
```

кекслет колледж

```
            const newTask = {
                id: tasks.length + 1,
                title: data.title,
                completed: false
            };

            tasks.push(newTask);

            res.statusCode = 201; // 201 означает "Успешно создано"
            res.end(JSON.stringify(newTask));
        } catch (err) {
            res.statusCode = 400;
            res.end(JSON.stringify({ error: 'Некорректный JSON' }));
        }
    });
}
```

хекслет колледж

```
    // 3. Обработка всех остальных запросов
    else {
        res.statusCode = 404;
        res.end(JSON.stringify({ message: 'Маршрут не найден' }));
    }
});

const PORT = 3000;
server.listen(PORT, () => {
    console.log(`API запущен на порту ${PORT}`);
});
```

хекслет колледж

POST http://localhost:3000/tasks

Params • Authorization Headers (8) Body • Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2     "title": "установить Potman"
3 }
```

Body Cookies Headers (5) Test Results 201 Created 5 ms 2

{} JSON Preview Visualize

```
1 {
2     "id": 2,
3     "title": "установить Potman",
4     "completed": false
5 }
```

екслет колледж

GET http://localhost:3000/tasks Ser

Params • Authorization Headers (8) Body • Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2     "title": "установить Potman"
3 }
```

Body Cookies Headers (5) Test Results 200 OK 7 ms 310 B

{} JSON Preview Visualize

```
1  [
2      {
3          "id": 1,
4          "title": "Выучить основы Node.js",
5          "completed": false
6      },
7      {
8          "id": 2,
9          "title": "установить Potman",
10         "completed": false
11     }
12 ]
```

екслет колледж

Важно: если мы остановим сервер (Ctrl+C), все новые задачи исчезнут. Это подводит нас к необходимости использования файлов или баз данных (тема будущих лекций).

Статус-коды как язык общения:

- 200 OK - просто отдали данные.
- 201 Created - подтвердили создание ресурса.
- 400 Bad Request - ошибка клиента (прислал пустую задачу).

хекслет колледж

Выводы:

- Роутинг - это фундамент любого веб-приложения. Без него сервер остается просто «автоответчиком». На чистом Node.js мы реализуем его через анализ строки req.url.
- GET и POST имеют принципиально разную природу в Node.js. GET-параметры доступны мгновенно в объекте URL, в то время как POST-данные нужно собирать по частям (чанками) через события.
- JSON - это стандарт. Современная веб-разработка практически полностью отказалась от передачи данных в виде простого текста или XML в пользу JSON.
- Безопасность парсинга. Использование try...catch при вызове JSON.parse() - это не «хороший тон», а обязательное условие стабильности сервера. Одна ошибка в JSON-строке клиента не должна убивать весь бэкенд.
- Статус-коды - часть интерфейса. Правильный выбор кода (201 вместо 200, 400 вместо 404) помогает фронтенд-разработчикам (и их скриптам) быстрее понимать, что произошло на сервере.

кекслет колледж

Контрольные вопросы

- Как извлечь значение параметра id из URL http://localhost:3000/user?id=10 ?
- В чем главная опасность использования req.url для прямого сравнения в switch, если мы ожидаем параметры?
- Что такое chunk в обработчике события req.on('data', chunk => ...) и какой у него тип данных по умолчанию?
- Какой заголовок Content-Type должен отправить сервер, если он возвращает массив объектов в формате JSON?
- Что произойдет с сервером Node.js, если выполнить JSON.parse('не-json-строка') без использования try...catch?
- В чем разница между JSON.parse() и JSON.stringify()?
- Какой HTTP-статус лучше всего подходит для успешного добавления новой задачи в To-Do List?

хекслет колледж